Chapter 2 - Summary and Sample Exam Questions

Why a VAE is needed

A deterministic autoencoder learns a mapping from an input x to a latent code z, and then back to a reconstruction $\hat{x}$. This is useful for compression and denoising, but it is **not** a principled generative model. The latent codes produced by a standard autoencoder can occupy arbitrary disconnected regions of latent space, so drawing a random latent vector and decoding it often produces unrealistic outputs. In other words, a deterministic autoencoder can reconstruct training-like examples, but it does not tell us what probability model generated the data.

A Variational Autoencoder fixes this by introducing a **probabilistic latent-variable model**. Instead of saying that each input maps to one exact latent vector, the encoder produces a **distribution** over latent variables. The decoder is also probabilistic: it defines a likelihood for the data conditioned on the latent variable. This gives the model a proper generative story.

The probabilistic model behind a VAE

A standard VAE assumes:

$$p(z) = \mathcal{N}(0, I)$$

and

$$p_\theta(x \mid z)$$

where $p(z)$ is the prior over latent variables and $p_\theta(x \mid z)$ is the decoder or generative model. The joint model is:

$$p_\theta(x, z) = p_\theta(x \mid z)\, p(z)$$

and the marginal likelihood of the observed data is:

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz$$

The goal is to learn θ so that $p_\theta(x)$ matches the true data distribution. The challenge is that the integral over z is generally intractable when the decoder is a neural network. This is the central computational problem of VAEs.

Why variational inference appears

If we could compute the true posterior

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)\, p(z)}{p_\theta(x)}$$

then learning and inference would be much easier. But this posterior depends on $p_\theta(x)$, which itself requires the intractable integral above. VAEs therefore introduce an approximate posterior, also called the encoder:

$$q_\phi(z \mid x)$$

In practice, this is usually chosen as a Gaussian with diagonal covariance:

$$q_\phi(z \mid x) = \mathcal{N}\big(\mu_\phi(x), \mathrm{diag}(\sigma^2_\phi(x))\big)$$

where the encoder network outputs $\mu_\phi(x)$ and $\log \sigma^2_\phi(x)$.

The key idea is not that $q_\phi(z \mid x)$ is the true posterior, but that it is a **tractable family** of distributions that can be optimized jointly with the decoder.

The ELBO and what it means

Because $\log p_\theta(x)$ is hard to optimize directly, the VAE maximizes a lower bound called the Evidence Lower Bound (ELBO):

$$\mathcal{L}_{\mathrm{ELBO}}(x) = \mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

This objective has two conceptually different parts.

The first term is the **reconstruction or likelihood term**. It encourages the latent variable inferred from x to contain enough information so that the decoder can explain or reconstruct x well.

The second term is the **regularization term**. It pushes the approximate posterior toward the prior. This makes the latent space smoother, more organized, and more sampleable. Without this pressure, the model could behave like a deterministic autoencoder with arbitrary latent codes.

The ELBO is not just a heuristic. It satisfies:

$$\log p_\theta(x) = \mathcal{L}_{\mathrm{ELBO}}(x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big)$$

Since the KL divergence is always nonnegative, the ELBO is indeed a lower bound on the log marginal likelihood. Maximizing the ELBO therefore does two things at once: it increases a tractable surrogate for model evidence, and it makes the approximate posterior closer to the true posterior.

Why reparameterization matters

The ELBO contains an expectation over samples from $q_\phi(z \mid x)$. If one samples z directly from a distribution whose parameters depend on ϕ, naive sampling breaks standard backpropagation. The reparameterization trick solves this by isolating the randomness in a parameter-free noise variable:

$$\epsilon \sim \mathcal{N}(0, I)$$

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$$

Now the stochasticity comes from ϵ, while z is a differentiable function of the encoder outputs. This lets gradients flow through the sample and enables efficient end-to-end learning with gradient descent.

Conceptually, reparameterization changes the question from "how do we differentiate through random sampling?" to "how do we express sampling as a deterministic transformation of fixed noise?" This is one of the core innovations that makes VAEs practical.

Geometry, information, and failure modes

A well-trained VAE learns a latent space in which nearby points tend to decode to similar outputs. This is what makes interpolation and random sampling meaningful. However, the latent space is shaped by a tension: too little regularization gives poor generative structure, while too much regularization can destroy information needed for accurate reconstruction.

This trade-off explains several important variants and failure modes.

In a β-**VAE**, the KL term is weighted more strongly:

$$\mathcal{L}_{\beta\text{-VAE}}(x) = \mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)] - \beta\, \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

When $\beta > 1$, the model is encouraged to use a more factorized and compressed latent code, which can promote disentanglement. But if β is too large, reconstructions degrade because the latent bottleneck becomes too restrictive.

A second extension is the **Conditional VAE (CVAE)**, which conditions both encoder and decoder on known information y:

$$q_\phi(z \mid x, y), \qquad p_\theta(x \mid z, y)$$

This is useful when part of the variation is known in advance, such as a class label. Conditioning reduces the burden on z: instead of encoding everything, the latent variables only need to model residual variation. This often improves controllability and reduces entanglement.

A common failure mode is **posterior collapse**, where:

$$q_\phi(z \mid x) \approx p(z)$$

for many or all inputs. In that case, z carries little information about x, and the decoder effectively ignores the latent variable. This often happens when the decoder is too powerful or when KL pressure is too strong early in training.

Summary

A VAE is best understood as a bridge between **deep learning** and **probabilistic inference**. It combines:

- a latent-variable generative model,
- approximate Bayesian inference through $q_\phi(z \mid x)$,
- and end-to-end neural optimization via reparameterization.

Its deeper conceptual significance is that representation learning is no longer only about compressing data into features. It becomes about learning a latent space that supports **inference**, **generation**, **sampling**, and **controlled manipulation**. The VAE therefore occupies an important place in modern machine learning: it is both a generative model and a framework for learning structured probabilistic representations.

Question 1

Why is the ELBO called a lower bound on $\log p_\theta(x)$, and under what condition does it become exact?

Sample Answer

The ELBO is called a lower bound because

$$\log p_\theta(x) = \mathcal{L}_{\mathrm{ELBO}}(x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big)$$

Since KL divergence is always nonnegative, it follows that

$$\mathcal{L}_{\mathrm{ELBO}}(x) \leq \log p_\theta(x)$$

for every datapoint.

The bound becomes exact when the approximate posterior matches the true posterior:

$$q_\phi(z \mid x) = p_\theta(z \mid x)$$

In that case the KL gap is zero. This shows that VAE learning is not only about learning a decoder; it is also about learning an inference model that makes the bound tight.

Question 2

Interpret the two main terms in the ELBO,

$$\mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)] \quad \text{and} \quad \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

and explain the tension between them.

Sample Answer

The expectation term rewards the model for explaining the observation well using latent values that are plausible for that same observation under $q_\phi(z \mid x)$. In practice, this plays the role of a probabilistic reconstruction objective.

The KL term regularizes the approximate posterior so that it does not drift too far from the prior. This encourages a more organized latent space that supports sampling and reduces the risk that each datapoint receives an isolated, arbitrary code.

The tension is that better reconstruction often pushes the encoder to store more information about x in the latent variable, while stronger KL pressure pushes the latent representation toward a simpler distribution. A VAE must balance fidelity to the data against structure and regularity in latent space.

Question 3

Explain the reparameterization trick conceptually. Why is it more than just a coding convenience?

Sample Answer

The reparameterization trick rewrites a latent sample as

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

This separates randomness from the learnable parameters. The noise is drawn from a fixed source, while the encoder controls how that noise is transformed into a latent sample.

Conceptually, this matters because it makes the stochastic step behave like a differentiable computation. Gradients can then flow through the latent sample back into the encoder parameters. Without this idea, sampling would interrupt ordinary backpropagation and make learning much harder.

So the trick is not merely implementation detail. It is one of the key ideas that makes variational autoencoders trainable with standard deep learning methods.

Question 4

A standard VAE does not automatically guarantee disentangled latent variables. Why not, and how do ideas such as β-VAE or CVAE try to change this?

Sample Answer

A standard VAE encourages reconstruction quality and approximate posterior regularity, but those pressures alone do not force separate latent coordinates to align with separate semantic factors. The model may still encode several factors together in entangled ways if that helps optimize the objective.

A β-VAE modifies the tradeoff by increasing the weight on the KL term:

$$\mathcal{L}(x) = \mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)] - \beta\, \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

When $\beta > 1$, the model is pushed toward a more constrained latent representation, which can encourage factor separation, although often at the cost of reconstruction quality.

A CVAE introduces side information such as labels or attributes. By explicitly conditioning on known factors, it reduces the burden on the latent variables and can produce a more structured representation. In both cases, the broader lesson is that disentanglement usually requires extra inductive bias rather than emerging automatically.

Question 5

Suppose a VAE encoder learns an approximate posterior that is almost identical to the prior for every input, so that

$$q_\phi(z \mid x) \approx p(z)$$

for all x. What does this suggest about how the model is using its latent variables, and what consequence would you expect for reconstruction quality?

Sample Answer

If $q_\phi(z \mid x)$ is nearly the same as $p(z)$ for every input, then the latent representation carries very little information about the specific datapoint x. In effect, the encoder is no longer distinguishing one input from another in a meaningful probabilistic way.

This usually means the decoder is being forced to reconstruct without relying much on the latent code. If the decoder is powerful enough, it may still produce reasonable outputs by learning generic patterns in the data, but reconstruction quality for individual examples will typically suffer because the latent variable is not providing input-specific information.

Conceptually, this reflects a failure to use the latent space as an informative hidden representation. The model may still optimize part of the objective, especially the KL term, but it is no longer achieving the intended balance between informative encoding and regularized generation.

Chapter 3 - Summary and Sample Exam Questions

The core idea of diffusion models

A diffusion model is a generative model that learns to create data by **reversing a gradual corruption process**. Instead of trying to generate a complicated object such as an image in one step, the model starts from a very simple distribution, usually Gaussian noise, and then iteratively denoises it. The central intuition is that it may be easier to learn many small denoising steps than one direct map from random noise to a realistic sample.

This gives diffusion models a very different flavor from classical latent-variable models. In a VAE, one introduces a low-dimensional latent variable z and decodes from it. In diffusion, the latent structure is instead a **trajectory**:

$$x_0 \to x_1 \to x_2 \to \cdots \to x_T$$

where x_0 is a clean data sample and x_T is approximately pure Gaussian noise.

The forward process as a Markov chain

The forward process, usually written as q, gradually destroys information in the data through a Markov chain:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t; \sqrt{\alpha_t}\, x_{t-1}, (1-\alpha_t) I\big)$$

with $\alpha_t = 1 - \beta_t$. Here β_t is the noise schedule, which determines how aggressively information is removed at each step.

This process is Markovian because each new noisy state depends only on the previous one. Over many steps, the distribution approaches something close to a standard Gaussian. The forward chain is **chosen by the designer** and is easy to sample from. It is not learned.

A key conceptual point is that the forward process is not just adding arbitrary noise. It defines a controlled path from structured data to unstructured noise. That path is what makes learning the reverse process possible.

Trajectory distributions versus marginals

In diffusion, it is essential to distinguish between the distribution of the **entire path** and the distribution of a **single noisy state**.

The trajectory distribution is:

$$q(x_{1:T} \mid x_0)$$

while the one-step-at-time marginal of interest is:

$$q(x_t \mid x_0)$$

The first object describes the full noising history. The second says what x_t looks like if we start from x_0 and jump directly to time t. This distinction matters because training usually uses the marginal $q(x_t \mid x_0)$ rather than simulating every intermediate step.

Because the forward process is Gaussian, the marginal also has a closed form:

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t) I\big)$$

where

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$$

This closed form is one of the most important practical simplifications in DDPM-style models.

Why the reverse process must be learned

If the forward process is known, then in principle the ideal reverse transition would be:

$$q(x_{t-1} \mid x_t)$$

However, this reverse conditional depends on the unknown data distribution, so it is not directly available in closed form for real data. This is the core modeling challenge.

Diffusion models therefore introduce a learned reverse process:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t)$$

where typically

$$p(x_T) = \mathcal{N}(0, I)$$

and each reverse step is parameterized by a neural network. Conceptually, the model learns how to undo a tiny amount of noise at every timestep.

Thus, the forward process is fixed and analytic, but the reverse process is learned from data.

What the neural network actually predicts

A subtle but very important question is: **what should the network output?**

In DDPM, a common choice is to train a network $\epsilon_\theta(x_t, t)$ to predict the Gaussian noise that was added. If the forward marginal is written as

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

then the network is trained with a simple mean squared error loss:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, \epsilon, t}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]$$

This seems surprisingly simple given the complexity of the generative process. The reason it works is that, under the Gaussian forward construction, predicting noise is closely tied to approximating the reverse denoising direction. It is also related to score estimation, since the score of a Gaussian-corrupted sample can be expressed in terms of the noise.

Different parameterizations are possible. One may predict ϵ, x_0, the score $\nabla_{x_t} \log q(x_t)$, or a velocity-like target. These are different views of closely related information, but their numerical stability and behavior can differ.

Why timestep conditioning is essential

The denoising task at small t is very different from the denoising task at large t. When t is small, x_t is only slightly corrupted and still contains a strong semantic signal. When t is large, x_t may be almost indistinguishable from noise. A single denoiser without time information would face an ambiguous problem.

For that reason, the neural network must be conditioned on the timestep t. In practice, this is often done using sinusoidal embeddings, which transform the discrete or continuous time index into a vector representation the network can use.

Conceptually, time conditioning tells the model **how much information should still be present** and therefore what kind of denoising operation is appropriate.

Why U-Nets are commonly used

For images, the reverse process requires both local detail and global structure. U-Nets are effective because they combine:

- downsampling paths that capture large-scale context,
- upsampling paths that reconstruct fine detail,
- skip connections that preserve spatial information across scales.

This makes them especially suitable for denoising problems. The architecture is not mandatory in theory, but it matches the multiscale nature of image generation extremely well.

The broader conceptual point is that diffusion is not just a probabilistic idea; it is also an **iterative inverse problem**, and U-Nets are strong inverse-problem solvers.

The role of the noise schedule

The schedule β_t or equivalently α_t controls how fast information is destroyed during the forward process. If noise is added too aggressively, the reverse problem may become unnecessarily difficult. If noise is added too slowly, training may be inefficient or the model may spend too many steps learning nearly redundant denoising tasks.

The schedule therefore shapes the signal-to-noise ratio over time. It determines how semantics are distributed across timesteps and influences both training stability and sample quality.

This is why linear and cosine schedules behave differently: they allocate information destruction differently across time.

DDPM versus DDIM

DDPM typically uses a stochastic reverse process. At each step, the model predicts denoising information and then sampling noise may be injected according to the probabilistic transition.

DDIM changes the **sampling procedure** while keeping the same training objective. It can be interpreted as constructing a more deterministic trajectory through latent space. This often allows much faster sampling with fewer steps.

The important conceptual insight is that training and sampling are not identical notions. A model trained with a DDPM-style objective can be sampled in different ways, and those choices affect speed, diversity, and determinism.

Summary

Diffusion models can be understood at several levels at once:

- as latent-variable models over a long trajectory,
- as learned denoisers,
- as approximate reverse-time stochastic processes,
- as score-based generative models under appropriate parameterization.

Their power comes from combining a simple forward corruption process with a flexible neural approximation to the reverse dynamics. The model never explicitly memorizes the data distribution in closed form. Instead, it learns how to move samples step by step from noise toward the data manifold.

This is why diffusion models are both statistically elegant and practically powerful: they transform generation into a sequence of manageable local predictions

Question 1

What is the conceptual difference between $q(x_{1:T} \mid x_0)$ and $q(x_t \mid x_0)$, and why is that distinction important in diffusion?

Sample answer

$q(x_{1:T} \mid x_0)$ is the probability distribution over the entire noising trajectory starting from the clean sample x_0. It is a distribution over a whole sequence of latent states. By contrast, $q(x_t \mid x_0)$ is only the marginal distribution of one particular noisy state at time t.

This distinction matters because training usually needs only a randomly chosen noisy observation x_t, not the full path. The existence of the closed-form marginal

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t) I\big)$$

lets us directly sample training pairs (x_0, x_t) efficiently. The full trajectory is conceptually important for understanding diffusion as a Markov process, but the marginal is what makes practical optimization easy.

Question 2

Why is the true reverse conditional $q(x_{t-1} \mid x_t)$ not directly usable in realistic diffusion modeling, even though the forward process is known?

Sample answer

Knowing the forward conditional $q(x_t \mid x_{t-1})$ does not automatically give us a usable form for the reverse conditional $q(x_{t-1} \mid x_t)$ on real data. The reverse distribution depends on the population-level distribution of x_{t-1} induced by the data. In other words, Bayes' rule introduces terms involving the unknown data distribution.

So although the local forward corruption rule is simple and Gaussian, the exact reverse direction is only analytically accessible in special cases. For natural images or other complex data, the reverse conditional is effectively intractable. This is why diffusion models learn a parametric approximation $p_\theta(x_{t-1} \mid x_t)$ from data rather than derive it exactly.

Question 3

Why does predicting the added noise ϵ make sense as a training target? Why is it not an arbitrary engineering trick?

Sample answer

Predicting ϵ is natural because under the forward marginal we can write

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$$

with $\epsilon \sim \mathcal{N}(0, I)$. Once x_t and t are given, the unknown part is precisely the noise realization that corrupted the clean sample. Estimating that noise tells the model how far the sample has been pushed away from the underlying data structure.

This target is also mathematically linked to denoising and score estimation. For Gaussian perturbations, the score can be written in a form proportional to the noise term. Therefore, predicting ϵ is a convenient surrogate for learning the local reverse direction. It is simple computationally, but it is also principled statistically.

Question 4

Why must the denoising network know the timestep t? Could one imagine a single denoiser that receives only x_t?

Sample answer

In general, x_t alone is not sufficient because similar-looking noisy observations may correspond to very different denoising regimes. Early timesteps retain most of the original signal, so denoising should be gentle and detail-preserving. Late timesteps may be nearly pure noise, so denoising requires a much more global and uncertain inference.

Without time information, the mapping from x_t to the appropriate denoising update becomes ambiguous. Conditioning on t resolves that ambiguity by telling the model the current signal-to-noise level. Conceptually, the timestep is part of the state of the reverse problem, not merely a bookkeeping variable.

Question 5

Why are diffusion models often described as both latent-variable models and score-based models? Are these descriptions contradictory?

Sample answer

These descriptions are not contradictory; they emphasize different mathematical viewpoints. Diffusion models are latent-variable models because the trajectory

$$x_{1:T}$$

acts as a collection of latent variables mediating the generation of x_0. The full model can be written as a joint distribution over these hidden states.

They are also score-based models because, under suitable parameterizations, the learned denoiser implicitly estimates score-like quantities such as gradients of log densities at noisy intermediate levels. This score information determines how to move samples from low-density noisy regions toward high-density data regions.

Thus, one viewpoint emphasizes probabilistic graphical structure, while the other emphasizes geometric denoising directions in data space. The two are compatible descriptions of the same underlying process.

Chapter 4 - Advanced Diffusion Concepts

This chapter extends the basic diffusion framework into a more mature theoretical view. The central idea is that diffusion models are not only denoisers; they are also estimators of a time-dependent probability geometry. At each noise level t, the model is implicitly learning how probability mass is arranged in the noisy data distribution $p_t(x)$, and how to move a sample toward regions of higher probability under that distribution.

Diffusion as variational learning and why MSE appears

Diffusion training can be derived from a variational objective. The log-likelihood is lower-bounded by an evidence lower bound (ELBO) that decomposes into reconstruction-like, prior-matching, and intermediate consistency terms. Once the forward process is chosen to be Gaussian and the reverse variance is fixed appropriately, the intermediate KL terms reduce to a simple error on the reverse mean. After reparameterization, this becomes the familiar noise-prediction loss:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,t,\varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t,t)\|^2\right].$$

So the commonly used MSE objective is not an arbitrary engineering trick. It is a tractable surrogate that emerges from the probabilistic structure of the model.

The score perspective

A key conceptual shift is to interpret diffusion through the score:

$$s_t(x) = \nabla_x \log p_t(x).$$

The score is the direction of steepest increase in log-density. It tells us how to move a noisy sample toward a more likely region of the data distribution at the current noise level. For a unimodal Gaussian, the score points toward the mean. For a multimodal distribution, it points toward the locally most plausible mode. This makes the score field a geometric description of how denoising should proceed.

In DDPM-style training, the network predicts noise, but this prediction is closely tied to the score through the denoising-score identity:

$$\nabla_{x_t} \log p_t(x_t) \approx -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\,\varepsilon_\theta(x_t,t).$$

This means the model is effectively learning a score field, even when trained as a noise predictor.

Reverse diffusion as drift plus stochasticity

The reverse process can be interpreted as a combination of a deterministic correction term and a stochastic term:

$$x_{t-1} = x_t + \beta_t s_t(x_t) + \sqrt{\beta_t} z, \qquad z \sim \mathcal{N}(0, I).$$

The drift term $\beta_t s_t(x_t)$ moves the sample toward higher-probability regions, so it plays the role of denoising. The random term $\sqrt{\beta_t} z$ is equally important: it preserves sampling behavior and prevents the process from collapsing into pure mode-seeking gradient ascent. This chapter therefore clarifies that reverse diffusion is neither mere denoising nor mere random sampling; it is a calibrated stochastic transport process.

Conditioning and guided diffusion

Unconditional diffusion models learn $p(x)$, but many applications require controlled generation from $p(x \mid y)$. The challenge is that conditioning must influence an entire reverse Markov chain rather than a single forward pass. The chapter presents two major solutions.

For classifier guidance, Bayes' rule gives:

$$\nabla_x \log p(x \mid y) = \nabla_x \log p(x) + \nabla_x \log p(y \mid x).$$

This suggests a guided score:

$$s_t^{\text{guided}}(x) = s_t(x) + w\,\nabla_x \log p_\phi(y \mid x, t),$$

where w controls guidance strength. The diffusion model contributes realism, while the classifier contributes condition alignment.

For classifier-free guidance (CFG), the model is trained to operate both with and without the condition, and sampling combines the two predictions:

$$\varepsilon_{\text{cfg}} = (1+w)\varepsilon_\theta(x_t,t,y) - w\,\varepsilon_\theta(x_t,t,\varnothing).$$

CFG avoids the need for an external classifier and has become especially important in modern conditional diffusion systems.

Why a noise-aware classifier is necessary

In classifier-guided diffusion, the classifier is queried on noisy states x_t, not only on clean samples x_0. A classifier trained only on clean images may be unreliable at large noise levels, causing poor gradients and unstable guidance. A noise-aware classifier therefore learns:

$$p_\phi(y \mid x_t, t),$$

with training inputs generated by the same forward diffusion process. This aligns the guidance signal with the actual states visited during reverse sampling.

Guidance is a tradeoff, not a free improvement

Increasing guidance strength often improves condition adherence, but it can also reduce diversity, amplify artifacts, and make the sampler brittle. The same idea applies to the number of diffusion steps, the noise schedule, classifier robustness, and conditioning-drop probability in CFG. The chapter emphasizes that conditional generation is always a balance among realism, diversity, controllability, and computational cost.

Evaluation and FID

The chapter also discusses Fréchet Inception Distance (FID), which compares real and generated samples in a deep feature space rather than pixel space. FID is useful because it captures differences in both mean and covariance of feature distributions:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \operatorname{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right).$$

Conceptually, a lower FID suggests that generated samples match the feature statistics of real data more closely. However, FID is still only a proxy. It depends on the feature extractor, sample size, and domain match, and unusual values can arise from numerical estimation issues. The broader lesson is that diffusion evaluation should not rely on a single metric.

Summary

The chapter's deeper contribution is that it links several viewpoints into one coherent picture:

- variational learning explains why training objectives take the form they do,
- score matching explains what the network is really learning,
- reverse diffusion explains how sampling transports noise back to data,
- guidance explains how control is inserted into that transport,
- and evaluation clarifies what it means for controlled generation to be successful.

Taken together, these ideas move diffusion from a procedural recipe to a principled probabilistic framework for generative modeling and controllable sampling.

Question 1

What is the theoretical basis of classifier guidance?

Sample Answer

Classifier guidance follows from Bayes' rule. Since

$$\log p(x \mid y) = \log p(x) + \log p(y \mid x) - \log p(y),$$

taking gradients with respect to x gives

$$\nabla_x \log p(x \mid y) = \nabla_x \log p(x) + \nabla_x \log p(y \mid x).$$

The term $\nabla_x \log p(x)$ is the unconditional score learned by the diffusion model, while $\nabla_x \log p(y \mid x)$ comes from the classifier. Adding them produces a guided score that encourages samples to remain realistic while becoming more aligned with the desired condition.

Question 2

Why is a noise-aware classifier preferable to a standard classifier in classifier-guided diffusion?

Sample Answer

The reverse chain queries the classifier on noisy states x_t, not just on clean data x_0. A classifier trained only on clean inputs may behave unpredictably when asked to evaluate heavily corrupted samples. In that case, its gradient can be weak, unstable, or semantically meaningless.

A noise-aware classifier is trained on pairs (x_t, t) produced by the same forward diffusion corruption process. As a result, it learns how class evidence appears across noise levels and produces gradients that are relevant to the actual reverse trajectory. This makes guidance more stable and conceptually consistent with the diffusion framework.

Question 3

What is the core idea behind classifier-free guidance, and why is random condition dropping essential?

Sample Answer

Classifier-free guidance trains a single diffusion model to produce both conditional and unconditional predictions. During training, the condition is randomly removed for some examples and replaced by a null token. This forces the model to learn both

$$\varepsilon_\theta(x_t, t, y)$$

and

$$\varepsilon_\theta(x_t, t, \varnothing).$$

At sampling time, the two predictions are combined so that the conditional component is amplified relative to the unconditional one. The random dropping step is essential because without it the model would never learn a true unconditional branch, and the subtraction-based guidance mechanism would have no meaningful reference point.

Question 4

Why does stronger guidance usually improve condition adherence but reduce diversity?

Sample Answer

Guidance works by amplifying the component of the reverse update that favors the desired condition. As the guidance scale increases, the sampler is pushed more aggressively toward condition-consistent regions. This usually improves alignment with the class, text prompt, or other conditioning signal.

However, the same amplification shrinks the effective set of plausible outputs, because trajectories that do not strongly support the condition are suppressed. The process becomes more concentrated, which often reduces diversity and can introduce artifacts if the guidance overwhelms the learned unconditional structure. Thus guidance strength controls a realism-versus-control-versus-diversity tradeoff rather than delivering a free improvement.

Question 5

What does FID measure conceptually, and what are its limitations when evaluating diffusion models?

Sample Answer

FID compares the distributions of real and generated samples in a deep feature space. It summarizes each feature distribution by a mean and covariance, then computes a Fréchet distance between the two Gaussians. Conceptually, it measures whether generated samples reproduce the central tendency and spread of real data in a semantically meaningful representation space.

Its limitations are equally important. It depends on the chosen feature extractor, assumes a Gaussian approximation in feature space, can be sensitive to sample size and numerical instability, and may not fully capture controllability or prompt alignment. Therefore, FID is useful but incomplete; it should be interpreted alongside task-specific and qualitative evidence.

Chapter 5 - Summary and Sample Questions

The main idea of flow matching

Flow matching treats generation as a **transport problem**. We begin with a simple base distribution p_0 such as Gaussian noise, and we want to transform it into the data distribution $p_1 = p_{\text{data}}$.

Instead of learning a latent posterior as in a VAE, or a score / denoiser as in diffusion, flow matching learns a **time-dependent velocity field**

$$v_\theta(x, t),$$

which defines the ordinary differential equation

$$\frac{dx}{dt} = v_\theta(x, t), \qquad t \in [0, 1].$$

If this vector field is learned well, then particles initialized at $x(0) \sim p_0$ move under the learned dynamics so that the final states $x(1)$ follow the data distribution.

A useful intuition is the **wind-field metaphor**:

- x is the current particle location,
- t is time,
- $v_\theta(x, t)$ tells the particle **which direction** to move and **how fast**.

So generation is no longer framed as gradual denoising, but as **integrating a learned transport ODE**.

Why flow matching can be easier to reason about than diffusion or VAEs

In diffusion, the network is tied to a stochastic forward process and often implicitly estimates a score-like object related to

$$\nabla_x \log p_t(x).$$

In a VAE, the model must handle an approximate posterior $q_\phi(z \mid x)$ and a decoder likelihood $p_\theta(x \mid z)$.

Flow matching shifts the viewpoint. It avoids explicit score estimation and avoids latent posterior approximation by turning the problem into **supervised vector-field regression**.

A synthetic trajectory is created between a start point $x_0 \sim p_0$ and an endpoint $x_1 \sim p_1$.
For the simplest linear bridge,

$$x_t = (1 - t)x_0 + tx_1,$$

the target instantaneous velocity is known analytically:

$$u_t = \frac{dx_t}{dt} = x_1 - x_0.$$

The model is then trained by regression:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left\|v_\theta(x_t, t) - u_t\right\|_2^2\right].$$

Conceptually, this is elegant because the training target is a **known velocity** along a chosen path rather than an unknown density derivative.

The continuity-equation perspective

A deep conceptual point is that flow matching is not just moving individual samples. It is modeling the evolution of an entire **probability distribution over time**.

If $p_t(x)$ denotes the density of particles at time t, and if particles move according to the velocity field $v(x, t)$, then the density evolution obeys the **continuity equation**

$$\frac{\partial p_t(x)}{\partial t} + \nabla \cdot \left(p_t(x) v(x, t)\right) = 0.$$

This equation says probability mass is neither created nor destroyed; it is only **transported** through space by the flow.
That is why flow matching is best understood as a **mass transport framework** rather than merely a pointwise regression trick.

Why Euler integration works at sampling time

Once v_θ is learned, we still need to solve the ODE numerically.
The simplest solver is Euler's method:

$$x_{n+1} = x_n + \Delta t\, v_\theta(x_n, t_n).$$

This works because the exact solution satisfies

$$x(1) = x(0) + \int_0^1 v_\theta(x(t), t)\, dt,$$

and Euler approximates the integral by a Riemann sum.
So sampling from a flow model is conceptually initialize from noise, then repeatedly apply small velocity-driven updates.

This creates a useful distinction from diffusion:

- diffusion often uses many reverse denoising steps tied to stochastic or deterministic reverse dynamics,
- flow matching samples by **integrating a learned ODE field**.

The importance of the bridge and the target velocity

Flow matching depends strongly on how intermediate states are constructed.
The simplest bridge is linear interpolation:

$$x_t = (1 - t)x_0 + tx_1,$$

But this is only one possible choice. More generally, one can define

$$x_t = I(x_0, x_1, t),$$

with target velocity

$$u_t = \frac{\partial}{\partial t} I(x_0, x_1, t).$$

This is important because the choice of bridge changes:

- what intermediate states the model sees,
- how difficult the velocity field is to learn,
- how straight or curved the transport trajectories become.

Thus, the learned vector field is not just a property of the data distribution; it is partly shaped by the **interpolation design**.

Latent flow matching

A major extension is **latent flow matching (LFM)**.
Instead of learning the transport in high-dimensional data space, we first learn an encoder–decoder system:

$$z = E_\phi(x), \qquad \hat{x} = D_\psi(z),$$

and then learn a velocity field in latent space:

$$v_\theta(z, t).$$

The motivation is that latent spaces can be lower-dimensional and semantically cleaner than pixel space.
In such a space, transport is often easier because nuisance detail has already been compressed away.

A standard latent flow matching setup is:

- choose z_1 from encoded data,
- choose $z_0 \sim \mathcal{N}(0, I)$,
- define

$$z_t = (1 - t)z_0 + tz_1,$$

- train on the target velocity

$$u_t = z_1 - z_0.$$

At generation time:

1. sample $z(0) \sim \mathcal{N}(0, I)$,
2. integrate the latent ODE,
3. decode $z(1)$ back to data.

This resembles latent diffusion in spirit, but the learned object is a **velocity field** rather than a score or noise predictor.

Conditional generation and guidance in flow matching

Flow matching can be made conditional by letting the velocity field depend on a label or condition y:

$$\frac{dx}{dt} = v_\theta(x, t, y).$$

This supports class-conditional generation. However, conditional training alone does not always give strong enough adherence to the desired label at sampling time.
This motivates **guidance**, especially **classifier-free guidance (CFG)**.

In flow matching CFG, the same model is trained to operate both:

- with the condition y,
- and without a condition $\varnothing$.

At sampling time, one constructs a guided velocity:

$$v_{\text{cfg}}(x, t, y) = v_\theta(x, t, \varnothing) + s\left(v_\theta(x, t, y) - v_\theta(x, t, \varnothing)\right),$$

where s is the guidance scale.

The conceptual meaning is:

- $v_\theta(x, t, \varnothing)$ provides a **realism baseline**,
- the difference term provides a **class-specific push**,
- s controls the tradeoff between fidelity to the condition and overall sample diversity / stability.

This mirrors CFG in diffusion, but the guided object here is a **velocity field** rather than a score estimate.

Classifier guidance and noise-aware classifiers

Another conditioning route is **classifier guidance**.
A separate classifier estimates something like $p(y \mid x_t, t)$, and its gradient with respect to the sample is used to bias the generative dynamics toward class y.

The important conceptual issue is that the classifier must work on **noisy or intermediate states**, not only clean data.
That is why a **noise-aware classifier** is often necessary: the conditioning signal must remain meaningful throughout the generation trajectory.

This highlights a broad principle across diffusion and flow-style models:

> Guidance only works well if the conditioning signal is valid at the intermediate states actually visited during sampling.

Optimal transport (OT) couplings

A particularly important refinement is to change not the bridge formula itself, but the **pairing** between x_0 and x_1.

In naive flow matching, the start and end points are paired independently.
In OT-based flow matching, one instead seeks a coupling

$$\pi^\star \in \arg\min_{\pi \in \Pi(p_0, p_1)} \mathbb{E}_{(x_0, x_1) \sim \pi}\left[\|x_0 - x_1\|^2\right].$$

This means the model learns from more coherent start–end pairings.
The practical effect is that the resulting transport paths are often shorter, less tangled, and easier to approximate.

So OT-based flow matching does not necessarily change the basic ODE view. Instead, it changes the **geometry of supervision** by changing which sample pairs define the training stories.

Consistency models and their relation to flow matching and diffusion

The chapter also connects flow ideas to **consistency methods**.
A consistency model learns a map

$$f_\theta(x_t, t)$$

that should be approximately invariant across noise levels for the same underlying clean sample:

$$f_\theta(x_s, s) \approx f_\theta(x_t, t), \qquad s < t.$$

The promise is extremely fast generation, often in one or a few steps.
But the conceptual challenge is that pure consistency objectives can be ill-posed. A trivial constant function can satisfy consistency without actually generating meaningful data.

This is why **teacher–student consistency** is important. A strong teacher model, often diffusion-based, provides a meaningful anchor so the student learns a nontrivial canonicalization map rather than collapsing.

Thus, the chapter places flow matching in a wider family of **transport-based generative models**:

- **VAE:** transport through a latent bottleneck and decoder,
- **Diffusion:** reverse-time stochastic or deterministic transport driven by denoising / score estimation,
- **Flow matching:** deterministic ODE transport through a learned velocity field,
- **OT flow matching:** flow matching with geometry-aware couplings,
- **Consistency models:** direct few-step transport, often distilled from a teacher.

Summary

The deepest conceptual lesson is that many modern generative models can be interpreted as learning different kinds of **transport mechanisms** from a simple distribution to a complex data distribution.

They differ mainly in the object they learn:

- VAE learns an encoder–decoder latent mechanism,
- diffusion learns a denoising / score-related mechanism,
- flow matching learns a velocity field,
- consistency models learn a time-consistent canonicalization map.

Chapter 5 is therefore not just about one algorithm. It is about a **family of related ways to move probability mass**, with different tradeoffs in tractability, controllability, stability, and sampling speed.

Question 1

Why is flow matching best understood as a transport model rather than simply a regression model?

Sample answer

At the surface level, flow matching is trained with supervised regression:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left\|v_\theta(x_t, t) - u_t\right\|_2^2\right].$$

So one might think it is "just" vector regression. But this view is incomplete.

The deeper object of interest is the induced dynamics

$$\frac{dx}{dt} = v_\theta(x, t),$$

which transports an entire probability distribution from p_0 to p_1.
The governing population-level equation is the continuity equation

$$\frac{\partial p_t}{\partial t} + \nabla \cdot (p_t v_t) = 0.$$

Thus, the regression target is only the training mechanism. The learned object is a **transport field** that moves probability mass. The real conceptual content of the method lies in the induced distributional dynamics, not merely in minimizing an MSE loss.

Question 2

What is the conceptual role of the interpolation path x_t, and why is it not a trivial design choice?

Sample answer

The interpolation path defines what the model will regard as a plausible intermediate state between base and data samples.
For a linear bridge,

$$x_t = (1 - t)x_0 + tx_1,$$

the target velocity is

$$u_t = x_1 - x_0.$$

But different bridges induce different intermediate geometries and different supervision signals. This changes:

- the distribution of training inputs (x_t, t),
- the target velocity seen at those inputs,
- the complexity of the velocity field required for accurate transport.

So the bridge is not a harmless implementation detail. It expresses a modeling assumption about how probability mass should move through state space. Better bridges can make learning easier and sampling more stable.

Question 3

Why can latent flow matching be preferable to pixel-space flow matching?

Sample answer

Pixel space is high-dimensional and contains large amounts of local detail, nuisance variation, and non-semantic structure.
Learning a velocity field directly in this space can require the model to capture too much low-level complexity.

Latent flow matching first defines a representation

$$z = E_\phi(x),$$

and then learns transport in the lower-dimensional latent space:

$$\frac{dz}{dt} = v_\theta(z, t).$$

This can help because:

- dimensionality is reduced,
- semantic structure is often more regular,
- transport trajectories may become simpler,
- computation can be cheaper.

However, the gain depends on the latent representation being good. If the autoencoder discards important information, then the decoder becomes a bottleneck and sample fidelity may be capped.

Question 4

Why do optimal-transport couplings matter even if the path formula remains linear?

Sample answer

Even if the bridge stays

$$x_t = (1 - t)x_0 + tx_1,$$

the choice of which x_0 is paired with which x_1 determines the geometry of the training trajectories.

Random pairings can create unnecessarily long or crossing trajectories, which in turn force the model to learn a more complex vector field.
OT couplings instead seek pairings that minimize expected transport cost:

$$\pi^\star \in \arg\min_{\pi \in \Pi(p_0, p_1)} \mathbb{E}_\pi\left[\|x_0 - x_1\|^2\right].$$

This often yields shorter, more coherent displacement directions and smoother supervision. So the path formula is only one ingredient; the **coupling** also shapes the difficulty of the learning problem.

Question 5

Why are consistency models conceptually attractive, and why are they also fragile?

Sample answer

Consistency models are attractive because they aim to map multiple noisy versions of the same underlying sample to a shared canonical output:

$$f_\theta(x_t, t) \approx f_\theta(x_s, s).$$

This suggests one-step or few-step generation, which is appealing from a sampling-speed perspective.

However, the objective is fragile because consistency alone does not guarantee correctness. A trivial constant function can satisfy the consistency relation without preserving data structure. In addition, ambiguous high-noise states can drive the model toward conditional means, causing mode averaging or collapse.

So consistency is powerful, but only when the objective is stabilized, usually by anchors, teacher guidance, or carefully designed training targets.

Chapter 6 - Summary and Sample Questions

Autoregressive modeling: the core probabilistic idea

An **autoregressive** model defines a probability distribution over a sequence by factorizing it into a product of conditional probabilities:

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

This equation is important because it turns the difficult task of modeling an entire sequence at once into a sequence of **next-step prediction** problems. In language modeling, for example, the model learns to predict the next token from the tokens that came before it. Training is therefore closely tied to the probabilistic structure of the model: learning to predict the next token well is the same as improving the likelihood of the training data.

This formulation also explains how generation works. At inference time, the model starts with an initial prompt, predicts a distribution over the next token, samples or selects one token, appends it to the context, and repeats. So the same factorization supports both **training** and **generation**.

2. RNNs and LSTMs: early sequence models

Before Transformers, sequence modeling was dominated by **Recurrent Neural Networks (RNNs)** and later **Long Short-Term Memory (LSTM)** networks. An RNN processes a sequence one step at a time, updating a hidden state:

$$h_t = f(h_{t-1}, x_t).$$

The hidden state acts as a running summary of the past. This is elegant, but it creates a bottleneck: all relevant earlier information must be compressed into a single evolving vector. As sequences become longer, this compression becomes harder, and learning long-range dependencies becomes unreliable.

LSTMs improve on this by introducing gated mechanisms that control what information is written, kept, and exposed. Conceptually, the gates let the model decide when to preserve older information and when to overwrite it. This helps with vanishing-gradient problems and makes longer dependencies more manageable than in plain RNNs. However, LSTMs still process tokens **sequentially**, which limits parallelism and can still make very long-range reasoning difficult.

Training recurrent models requires **Backpropagation Through Time (BPTT)**. Since the same recurrent computation is reused across many time steps, gradients must be propagated through a long chain of dependencies. This makes optimization sensitive to exploding or vanishing gradients and highlights an important lesson: even when the model definition is simple, the optimization path may be difficult.

Why attention changes the picture

The central idea behind the Transformer is that a token should not rely only on a compressed hidden state from the immediate previous step. Instead, it should be able to look back at the entire relevant context and determine **which earlier tokens matter most** for constructing its current representation.

This is the role of **attention**. In self-attention, each token produces three vectors:

- a **query** that represents what information the token is looking for,
- a **key** that represents what kind of information each token offers,
- a **value** that contains the content to be aggregated.

The compatibility between token i and token j is measured by a score such as:

$$s_{ij} = \frac{q_i k_j^T}{\sqrt{d_k}}.$$

These scores are normalized with a softmax to produce attention weights, and the output for token i becomes a weighted combination of value vectors:

$$\alpha_{ij} = \mathrm{softmax}_j(s_{ij}),$$

$$out_i = \sum_j \alpha_{ij} v_j.$$

This gives each token a **contextualized representation**. A word is no longer represented only by its standalone embedding, but by a representation that depends on the surrounding tokens and the task of the current layer.

The use of separate query, key, and value projections is conceptually important. The model not only decides **where to look** but also **what content to retrieve** from those locations. Attention is therefore more than a weighted average; it is a learned content-based retrieval mechanism.

The Transformer block

A Transformer does not consist of attention alone. A standard block usually contains:

1. **Multi-head self-attention**
2. A **position-wise feed-forward network (FFN)**
3. **Residual connections**
4. **Layer normalization**

Multi-head attention means the model runs several attention mechanisms in parallel, each with different learned projections. This allows the model to capture different kinds of relationships at the same time, such as short-range syntactic cues, long-range semantic references, or positional patterns.

The **FFN** is applied independently to each token after attention. Attention mixes information **across tokens**, while the FFN transforms information **within each token representation**. This combination is powerful: attention handles interaction across positions, and the FFN increases local representational capacity.

Residual connections help preserve information and support gradient flow, while **LayerNorm** stabilizes training by normalizing features within each token representation. Together, these design choices make deep Transformer stacks trainable in practice.

Because self-attention compares tokens directly, Transformers avoid the single-state bottleneck of RNNs and allow much greater parallelism during training. This is one of the main reasons they became dominant.

Positional information and causal structure

Self-attention by itself is permutation-invariant: without additional structure, it does not know which token came first or last. That is why Transformers need **positional information**, added through positional encodings or learned positional embeddings.

For **decoder-only** language models such as GPT, attention must also obey the autoregressive constraint: token t may only attend to tokens up to position t, not future tokens. This is enforced by **causal masking**. Without such masking, the model could cheat during training by directly inspecting future tokens it is supposed to predict.

Tokenization and BPE

Language models do not usually operate on raw words. Instead, they operate on **tokens** drawn from a finite vocabulary. One practical challenge is that word-level vocabularies are too rigid and character-level models can make sequences unnecessarily long. **Byte-Pair Encoding (BPE)** provides a compromise by learning frequent subword units.

BPE starts from smaller units and repeatedly merges common symbol pairs. This allows the tokenizer to represent frequent words compactly while still handling rare or unseen words through smaller pieces. Conceptually, tokenization is not a minor preprocessing step; it changes the length of sequences, the granularity of meaning, the efficiency of training, and the kinds of patterns the model can learn.

GPT-style training and the larger Transformer family

A GPT-style model is a **decoder-only Transformer** trained with the next-token objective. Given an input prefix, the model outputs logits for the next token at every position. A softmax converts logits into probabilities, and training minimizes cross-entropy against the true next token. This is directly aligned with the autoregressive factorization of sequence probability.

This gives rise to a broader family of Transformer architectures:

- **Encoder-only** models learn bidirectional contextual representations and are strong for understanding tasks such as classification, tagging, and retrieval.
- **Decoder-only** models are naturally suited for generation because they are trained autoregressively.
- **Encoder–decoder** models separate source encoding from target generation and are especially useful for sequence-to-sequence tasks such as translation and summarization.

The broader lesson is that modern sequence modeling is not just about replacing one neural network with another. It reflects a shift in how context is represented: from a compressed sequential state to a flexible, learned pattern of content-based interactions across tokens.

Final Exam Conceptual Questions with Sample Answers

These questions are designed to be **self-contained**, **conceptual**, and suitable for an advanced final exam. They emphasize understanding, interpretation, and synthesis rather than programming.

Question 1

An autoregressive model writes

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

Explain why this is both a probabilistic statement and a practical training strategy.

Sample Answer

It is a probabilistic statement because it expresses the joint probability of the whole sequence in terms of conditional distributions. The model is therefore assigning likelihood to complete sequences, not merely making isolated predictions.

It is also a practical training strategy because maximizing the joint likelihood becomes equivalent to training the model to predict each next token from its prefix. This converts sequence modeling into a large collection of supervised prediction problems, one at each position in the sequence.

The factorization also explains generation: once the conditionals are learned, the model can generate by repeatedly predicting and appending the next token.

Question 2

In self-attention, why do we need separate query, key, and value projections? Why not let one vector play all three roles?

Sample Answer

The query represents what a token is currently looking for, the key represents what each token offers for matching, and the value represents the content to be retrieved once a match is made. Keeping these roles separate gives the model more expressive flexibility.

If one vector had to serve all three roles, the model would have less freedom to distinguish matching behavior from content storage. Separate projections allow the model to learn one geometry for deciding relevance and another for representing information to be passed forward.

This separation is one reason attention can behave like learned retrieval rather than simple similarity averaging.

Question 3

The output of attention is a weighted sum of value vectors. Why is the value vector conceptually necessary once the model has already computed relevance weights?

Sample Answer

The relevance weights only indicate which positions matter and by how much. They do not specify what information should actually be extracted from those positions.

The value vectors provide the content that is aggregated after the relevance pattern has been decided. This separation means a token can attend strongly to a position because it is relevant, while still retrieving a transformed representation of the information stored there.

Without values, attention would know where to look but not have a flexible mechanism for deciding what representation to pull back from that location.

Question 4

Why can multi-head attention be more expressive than single-head attention even if both operate on the same input sequence?

Sample Answer

Different heads use different learned projections, so they can focus on different types of relationships at the same time. One head might track local syntactic structure, another might follow long-range references, and another might attend to positional or semantic patterns.

A single head must compress all these possibilities into one attention map and one value aggregation process. Multi-head attention distributes the work across parallel subspaces, allowing the model to represent richer interaction patterns.

The advantage is not just more parameters, but structured diversity in how context is analyzed.

Question 5

Why is causal masking essential in a decoder-only language model? What would happen conceptually if it were removed during training?

Sample Answer

A decoder-only language model is supposed to model

$$p(x_t \mid x_{<t}).$$

Causal masking enforces this by preventing token t from attending to future tokens. If the mask were removed during training, the model could directly use the answer token it is supposed to predict.

This would make training deceptively easy but would destroy the intended probabilistic interpretation. The model would no longer be learning a valid autoregressive conditional distribution, and the resulting training objective would not match generation-time constraints.

Chapter 7 - Summary and Sample Questions

Why Vision Transformers matter

A Vision Transformer (ViT) applies the transformer idea to images by treating an image as a **sequence of visual tokens**. The central move is to divide the image into patches and process those patches with self-attention in much the same way that a language model processes word tokens.

ViTs matter for two reasons. First, they showed that strong image understanding does not require convolution to be the only organizing principle. Second, they became a natural bridge from image classification to modern **vision-language** and **generative** systems. The same encoder idea can feed a classifier, a contrastive objective, a multimodal alignment module, or a generative backbone.

From image to token sequence

Suppose an input image has spatial size $H \times W$ and is split into non-overlapping patches of size $P \times P$. Then the number of patches is

$$N = \frac{H}{P}\frac{W}{P}.$$

Each patch is flattened and mapped into an embedding vector in $\mathbb{R}^D$. This gives a patch-token matrix

$$X_{\text{patch}} \in \mathbb{R}^{N \times D}.$$

So the key abstraction is that an image becomes a sequence of N learned token vectors. This is conceptually powerful because it lets a transformer reason over the image using the same attention machinery developed for sequences.

Patch embedding as linear projection or convolution

Patch embedding can be understood in two equivalent ways. One view is that each flattened patch is passed through a learned linear map into $\mathbb{R}^D$. Another view is that a convolution with kernel size P and stride P simultaneously **extracts non-overlapping patches** and **projects them** into D channels.

This is important conceptually because the patch embedding stage is the point where raw pixels are converted into token representations suitable for attention. It is not yet "understanding" the image at a high semantic level; it is creating the token interface on which the transformer operates.

Positional information is essential

A transformer is not inherently aware of spatial arrangement. If the model only receives patch content and no positional information, then it cannot reliably distinguish between different spatial layouts containing the same set of patches. For this reason, positional embeddings are added:

$$X' = X + P,$$

where P is a learned or fixed positional table.

In vision, positional encoding is especially important because images are spatial objects. A design choice arises: whether to use flattened 1D positions, explicit 2D structure, or relative positional information. The choice affects how well the model handles translation, spatial relations, and changes in resolution.

The class token and image-level prediction

For classification, a learnable class token is typically prepended to the patch sequence:

$$X_0 = [x_{\text{cls}}; x_1; x_2; \ldots; x_N].$$

After passing through the encoder, the final class-token representation is used as a global image summary:

$$\text{logits} = W_{\text{cls}} x_{\text{cls}}^{(L)} + b.$$

The class token is useful because it gives the network a dedicated location in the sequence for accumulating image-level information through self-attention. In bidirectional attention, the class token can attend to every patch and every patch can influence the class token.

Transformer encoder blocks in ViT

A ViT uses stacked transformer encoder blocks. Each block contains:

1. Layer normalization
2. Multi-head self-attention
3. Residual connection
4. Layer normalization
5. MLP
6. Residual connection

The attention operation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V.$$

This means that each token can dynamically gather information from other tokens according to learned relevance scores. In vision, this allows patches from distant image regions to interact directly, which is one reason ViTs can model long-range relationships effectively.

Classification versus general-purpose encoding

A ViT encoder can serve different tasks. For classification, the output head maps the final class-token state to class logits. For representation learning or multimodal learning, the encoder instead produces a general image embedding that may come from the class token, mean pooling, or another pooling mechanism.

So the encoder is often the stable core, while the **task head** changes. This is why ViTs are so reusable across settings such as classification, retrieval, CLIP-style image-text alignment, and downstream fine-tuning.

Strengths and limitations of ViTs

ViTs have several conceptual strengths. They provide flexible global interaction through attention, align naturally with transformer-based multimodal systems, and can scale effectively with data and compute. They also offer a unified architecture that works well across classification, representation learning, and generative modeling.

However, they also have important limitations. Self-attention scales quadratically with sequence length, so making patches too small can become expensive because N grows rapidly. ViTs also do not build in the same strong local inductive biases as CNNs. As a result, they often depend more heavily on data, augmentation, pretraining, or architectural refinements.

CNNs versus ViTs

CNNs build in locality, translation-related biases, and hierarchical feature extraction through convolution and pooling. ViTs replace most of that hard-wired structure with learned token interactions. This leads to a trade-off.

CNNs can be more data-efficient and naturally aligned with local image structure. ViTs can be more flexible and often stronger when trained at scale or integrated into multimodal and generative pipelines. Modern practice increasingly shows that the two ideas are not strict opposites; many successful systems mix convolutional and transformer-style design choices.

ViTs beyond classification

Vision Transformers are not only classifiers. They are now central in masked-image modeling, multimodal encoders, and image generation. In diffusion-style systems, transformer backbones can replace CNN-style U-Nets or operate in latent/tokenized spaces. This shift matters because it suggests that the transformer is not just a language architecture adapted to vision; it has become a general-purpose design pattern for perception and generation.

A useful conceptual summary is this:

$$[B, C, H, W] \to [B, N, D] \to [B, N+1, D] \to [B, D] \to [B, K].$$

The image becomes patch tokens, a class token is added, the encoder contextualizes the sequence, and a task-specific head turns the resulting representation into the desired output.

Final Exam Conceptual Questions with Sample Answers

Question 1

Consider a Vision Transformer without any class token. Is image classification still possible? Explain conceptually how and discuss one trade-off.

Sample Answer

Yes, classification is still possible. The model can aggregate the final patch tokens using mean pooling, attention pooling, or another learned global summarization method, and then apply a classifier to that global representation.

The trade-off is that without a class token, the model loses a dedicated sequence position whose role is to accumulate task-specific global information. Pooling may be simpler and more symmetric, but it may also be less explicitly specialized for classification than a learned class token pathway.

Question 2

Why do residual connections and layer normalization play an important role in a deep Vision Transformer, even though neither one directly adds visual knowledge?

Sample Answer

They play an optimization and stability role. Residual connections help preserve information across layers and make it easier to train deep stacks without destroying earlier representations. Layer normalization stabilizes the scale of activations, which makes learning more predictable.

Although they do not encode image structure directly, they are crucial for turning the transformer from a conceptual architecture into a trainable deep system. Without such mechanisms, learning can become unstable or inefficient.

Question 3

A student claims that ViTs are simply better than CNNs because they are newer and more general. Critically evaluate this claim.

Sample Answer

The claim is too simplistic. ViTs are highly flexible and have become central in many modern systems, especially when large-scale pretraining or multimodal integration is involved. They can capture global interactions elegantly and transfer well across tasks.

However, CNNs remain strong because their inductive biases are well matched to many visual problems. They can be more efficient, more data-frugal, and highly competitive when modernized. The better conclusion is that ViTs and CNNs offer different advantages, and performance depends on data scale, compute budget, and task structure.

Question 4

Why is the rise of transformer backbones in generative image models conceptually significant, rather than just an engineering curiosity?

Sample Answer

It is significant because it suggests that the transformer has become a general representation mechanism rather than a tool tied only to language. If transformer-style backbones can support both understanding and generation, then the same broad architectural principles may unify multiple kinds of perception and synthesis.

This also changes how one thinks about model design. Instead of treating classification backbones and generative backbones as fundamentally separate families, one can view them as different uses of related token-based architectures.

Question 5

Summarize the full conceptual flow of a Vision Transformer from raw image to output prediction, and identify the main design choices at each stage.

Sample Answer

The image first enters as a tensor of pixels and is split into patches. Those patches are projected into token embeddings, often using a linear map or a convolutional implementation. Positional information is added so the model knows where tokens came from, and optionally a class token is prepended for image-level prediction.

The sequence then passes through stacked transformer encoder blocks, where self-attention and MLP layers repeatedly refine the token representations. Finally, an aggregation choice is made, such as reading the class token or pooling patch tokens, and a task-specific head maps the representation to class logits or another target space.

The main design choices include patch size, embedding dimension, positional encoding type, use of a class token, encoder depth, number of attention heads, and the form of the output head.

Chapter 8 - Summary and Sample Questions

What a Vision–Language Model is trying to learn

A Vision–Language Model (VLM) learns a joint relationship between **images** and **language**. The main goal is not simply to classify an image or generate text in isolation, but to connect **visual meaning** and **linguistic meaning** in a useful shared framework. A strong VLM should know that an image of a dog, the phrase *"a dog"*, and a caption such as *"a brown dog running on grass"* are all semantically related, even though they are expressed in different modalities.

This matters because many important tasks are naturally multimodal: image retrieval from text, caption generation, visual question answering, grounding language in images, and zero-shot recognition.

The two dominant architecture families

A useful way to organize VLMs is to separate them into two broad families.

Dual-encoder contrastive models

In a dual-encoder model, the image and the text are processed by **separate encoders**. The image encoder produces an embedding v, and the text encoder produces an embedding t. The model is trained so that matched image-text pairs are close and mismatched pairs are far apart. A common similarity score is cosine similarity on normalized embeddings:

$$s(x, y) = \hat{v}^\top \hat{t}.$$

This family is represented by **CLIP-like** models.

Its main strength is that it learns a **shared embedding space**. Once this space is learned, the model can support:

- **zero-shot classification**, by comparing an image to prompts such as *"a photo of a cat"* or *"a photo of a dog"*
- **text-to-image retrieval**, by ranking images according to similarity with a query
- **image-to-text retrieval**, by ranking candidate texts for an image

The main limitation is that the model usually learns mostly **global alignment**, not detailed token-by-token reasoning. It can know that an image and sentence belong together without explicitly modeling fine spatial relations or generating fluent text token by token.

Fusion-based encoder–decoder models

A fusion-based VLM more directly combines visual and textual information. The image is encoded into a set of visual tokens or memory vectors, and the text side uses a decoder that can attend to those visual representations through **cross-attention**.

This family is natural for tasks such as:

- caption generation
- visual question answering
- grounded multimodal generation

Its strength is that it can condition each generated text token on the image. Its limitation is that it is often more expensive, more sequential at inference time, and less naturally suited to large-scale retrieval than a pure dual encoder.

The CLIP idea: alignment by contrastive learning

CLIP learns from matched image-text pairs. Suppose a minibatch contains matched pairs (x_i, y_i). The model computes similarities between every image embedding and every text embedding, giving a score matrix:

$$s_{ij} = \frac{\hat{v}_i^\top \hat{t}_j}{\tau}.$$

Here τ is a **temperature** parameter. The temperature controls how sharp the similarity distribution is. A smaller temperature makes the model focus more strongly on separating the best matches from the rest.

The contrastive objective says: for each image i, the correct text i should receive high probability among all candidate texts in the batch, and symmetrically for each text. So the batch itself provides **negative examples**. This is conceptually important: CLIP does not only learn what matches, but also what should not match.

The result is not a classifier head tied to a fixed set of classes. Instead, the model learns a semantic space in which new text prompts can be used at test time.

Why zero-shot classification works

In standard supervised classification, the output layer is tied to a fixed label set. In CLIP, classification is reframed as **similarity matching**. To classify an image, one writes one or more prompts per class, embeds those prompts, embeds the image, and chooses the class whose text embedding has the highest similarity to the image embedding.

So zero-shot classification works because the model has already learned to align images with language. The class names are introduced at test time through natural-language prompts, not through a dedicated trained classifier head.

This is powerful, but prompt wording matters. Different prompt templates can change performance because the text encoder is sensitive to phrasing.

Why cosine similarity and normalization are important

When embeddings are normalized, the dot product becomes cosine similarity. This means the score focuses on **direction in semantic space** rather than raw vector magnitude. That is useful because magnitude can be influenced by factors that do not directly reflect semantic compatibility.

Normalization helps stabilize the geometry of the embedding space and makes similarity comparisons across examples more meaningful.

Retrieval versus generation

A dual-encoder model like CLIP is excellent for **retrieval** because both images and texts can be embedded independently and compared efficiently. This makes large-scale nearest-neighbor search practical.

However, CLIP does **not** directly generate language token by token. It says how compatible an image and a text are, but it does not itself define an autoregressive distribution over output words. That is why CLIP is naturally suited to retrieval and zero-shot recognition, while caption generation usually requires a **generative decoder** or another model on top of CLIP-like representations.

Fusion models: self-attention, cross-attention, and decoding

In a fusion encoder–decoder VLM, the image encoder first creates a set of visual features or memory vectors. The text decoder then performs two kinds of attention:

- **masked self-attention** over previously generated text tokens
- **cross-attention** from text tokens to visual memory

Conceptually, self-attention builds a contextual representation of the text prefix, while cross-attention allows the text representation to look into the image.

If the decoder is generating a caption, then at time step t it predicts:

$$p(y_t \mid y_{<t}, x).$$

This factorization makes the model genuinely generative. The image affects the output through cross-attention, while the previously generated tokens affect it through masked self-attention.

Why fusion can be better for detailed grounding

Some tasks require more than global compatibility. For example:

- counting objects
- describing spatial relations
- reasoning about attributes tied to specific regions
- answering questions that depend on localized evidence

A fusion model is often better suited to these tasks because language tokens can attend directly to visual memory. In other words, the model can condition the generation of each word on particular parts of the image.

By contrast, a pure dual encoder compresses the image and text into global vectors, which can be powerful but may lose fine-grained relational detail.

Limitations and failure modes

VLMs inherit several important weaknesses.

Bias and spurious correlations

If training data contain repeated shortcuts, the model may associate concepts with backgrounds, demographics, or stylistic artifacts rather than with the intended object or relation.

Weak grounding

A model may appear correct at a global level but still fail on local detail, counting, or spatial reasoning.

Distribution shift

Performance may degrade when the domain changes, such as moving from web images to medical, aerial, underwater, or multilingual settings.

Hallucination

Generative multimodal models may produce plausible descriptions that are not actually supported by the image.

Scaling, variants, and efficient VLMs

Later work keeps the CLIP-style interface but changes how it is trained or adapted. Some approaches alter the loss function, some lock one encoder and tune the other, and some focus on prompt learning or lightweight adapters. The key idea is that the **embedding space interface** remains useful even when the training recipe changes.

For deployment, smaller models such as TinyCLIP or MobileCLIP must trade off **accuracy, latency, and memory**. A smaller model is not just a compressed version in a trivial sense: as capacity shrinks, the model may lose robustness, long-tail semantic coverage, or sensitivity to nuanced wording. That is why **distillation** is often used.

In distillation, a small student tries to mimic a large teacher, not only in final predictions but often in embedding geometry. This is important because the quality of a VLM is not only about top-1 accuracy, but also about whether its semantic space preserves meaningful relationships across images and texts.

Summary

The chapter's central idea is that VLMs can be understood through two complementary views:

- **alignment models**, which learn a joint space for matching images and language
- **fusion/generative models**, which use visual information directly while generating text or answering multimodal queries

The first view emphasizes **semantic compatibility and transfer**. The second emphasizes **conditional generation and grounded reasoning**. Modern multimodal systems often combine ideas from both.

Final Exam Conceptual Questions with Sample Answers

Question 1

Why is the CLIP training objective fundamentally about both **positive pairs** and **negative pairs**? Why would training only on matched pairs be insufficient?

Sample Answer

CLIP must learn not only that a correct image-text pair should be similar, but also that incorrect pairs should be less similar. If the model only increased similarity for matched pairs without contrasting them against mismatched alternatives, the embedding space could collapse toward trivial solutions in which many unrelated examples also look compatible. The negative pairs define the geometry of separation. They teach the model what distinctions matter. This is why the batch structure is important: each batch provides both the correct match and a set of competing alternatives.

Question 2

In a CLIP-like model, why is **temperature** more than a cosmetic scaling constant? Explain its conceptual effect on learning.

Sample Answer

Temperature controls how sharply the model distinguishes the best match from the rest. A lower temperature makes similarity differences more consequential, which can strengthen discrimination but may also make optimization harsher or more sensitive. A higher temperature smooths the distribution and weakens the pressure to separate close competitors. Conceptually, temperature shapes how strongly the learning process emphasizes ranking precision among candidate pairs. So it directly affects the contrastive geometry the model learns.

Question 3

Why does zero-shot classification in CLIP not require a conventional classifier head trained on the target classes?

Sample Answer

Zero-shot classification works because CLIP reframes classification as language-conditioned similarity matching. Instead of learning a fixed classifier head with one weight vector per class, the model compares an image embedding to text embeddings built from prompts such as *"a photo of a cat"* or *"a photo of a truck."* The chosen class is the one whose prompt is most similar to the image. The knowledge of class meaning is therefore mediated through the text encoder and the learned joint embedding space, not through a special task-specific output layer.

Question 4

In a fusion encoder–decoder VLM, what is the conceptual difference between **masked self-attention** in the text decoder and **cross-attention** to the image representation?

Sample Answer

Masked self-attention lets the decoder build a contextual representation of the text prefix while respecting the autoregressive constraint that future tokens are not visible. It answers the question: *What has already been said?* Cross-attention, by contrast, lets the decoder consult the visual memory produced by the image encoder. It answers the question: *What in the image is relevant to the current word being generated?* So self-attention organizes linguistic context, while cross-attention injects visual evidence.

Question 5

During caption training, teacher forcing is often used. Explain why teacher forcing is helpful during optimization, and also describe one conceptual weakness of relying on it.

Sample Answer

Teacher forcing is helpful because, during training, the decoder receives the correct previous tokens rather than its own imperfect predictions. This makes optimization more stable and allows the model to learn the conditional next-token distribution efficiently. However, it creates a mismatch between training and inference. At test time, the model must condition on its own generated tokens, so earlier mistakes can accumulate. This gap is a conceptual weakness often described as exposure bias.

Chapter 9 - Summary and Sample Questions

Concept-focused summary of training and adapting vision-language models

Chapter 9 presents vision-language model adaptation as a **ladder of interventions** applied to a common autoregressive policy. A VLM is written as

$$p_\theta(y \mid x, v) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x, v),$$

where x is the text instruction, v is the visual input, and y is the generated response. This factorization is deliberately kept fixed across the chapter so that each method can be understood by asking one clean question: **what exactly is being changed?**

The chapter's central conceptual distinction is between methods that change only the **conditioning context** and methods that change the **policy parameters**. Zero-shot and few-shot prompting keep θ fixed. Zero-shot changes only the prompt, while few-shot inserts demonstration examples into the context window. The key idea is that behavior can still change dramatically without gradient descent because the transformer's hidden states depend on the full context. So few-shot learning is not parameter learning; it is **context-conditioned temporary adaptation**.

RAG adds a second kind of inference-time adaptation by introducing an **external memory**. Instead of depending only on pretrained knowledge or context examples, the model retrieves documents and conditions generation on them. In probabilistic form,

$$p_{\theta,\eta}(y \mid x, v) = \sum_{z \in Z} p_\eta(z \mid q(x, v))\, p_\theta(y \mid x, v, z).$$

This makes RAG a latent-variable model over retrieved evidence. The chapter emphasizes that the key object is not just the retrieved top-k list, but the posterior responsibility weight

$$w(z) = \frac{p_\eta(z \mid q(x, v))\, p_\theta(y^* \mid x, v, z)}{\sum_{z'} p_\eta(z' \mid q(x, v))\, p_\theta(y^* \mid x, v, z')},$$

which tells us which document actually helped explain the gold answer. This is important because it separates **retrieval plausibility** from **explanatory usefulness**.

The chapter then moves from inference-time adaptation to **parameter learning**. In supervised fine-tuning, the model is trained on labeled triples (x_i, v_i, y_i^*) and optimized by next-token maximum likelihood:

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\sum_{i=1}^{N} \sum_{t=1}^{T_i} m_{i,t} \log p_\theta(y_{i,t}^* \mid y_{i,<t}^*, x_i, v_i).$$

Here the mask $m_{i,t}$ determines which tokens count toward the loss. The chapter stresses an important multimodal instruction-tuning idea: in chat-style training, one often wants to predict only the **assistant span**, not the user or system text. So the practical question is not merely "what is the target answer?" but also "which token positions should define the learning signal?"

LoRA and QLoRA are presented as changes in **parameterization**, not changes in the task objective. LoRA keeps the pretrained weight matrix W frozen and learns a low-rank update

$$W' = W + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA.$$

This reduces trainable parameters from $d_{\text{out}} d_{\text{in}}$ to roughly $r d_{\text{in}} + d_{\text{out}} r$. The key conceptual lesson is that SFT and LoRA are not competing methods at the same level. SFT defines **what objective is optimized**, while LoRA defines **which parameters are allowed to move and in what form**. The chapter also explains why adapting attention projections such as `q_proj`, `k_proj`, `v_proj`, and `o_proj`, together with MLP projections such as `gate_proj`, `up_proj`, and `down_proj`, changes both information routing and nonlinear transformation.

The alignment part of the chapter begins with **preference tuning** through DPO. Instead of imitating one gold answer, DPO compares a preferred response y_w with a rejected response y_l and encourages the policy to increase the preferred answer's relative log-probability compared with a frozen reference policy:

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\log \sigma\left(\beta \left[\log \pi_\theta(y_w \mid x, v) - \log \pi_\theta(y_l \mid x, v) - \log \pi_{\mathrm{ref}}(y_w \mid x, v) + \log \pi_{\mathrm{ref}}(y_l \mid x, v)\right]\right).$$

The conceptual appeal is that DPO performs preference alignment without explicit online rollouts or PPO-style advantage estimation. But the chapter also warns that **relative preference is not identical to factual correctness**. If the preference pairs reward style more than truth, the model may become better aligned to human taste while becoming worse grounded scientifically.

The last stage reframes VLM adaptation as reinforcement learning. A state is the prompt, image, and generated prefix; an action is the next token; a trajectory is the full answer. The classical policy-gradient objective is written in terms of expected trajectory reward, and PPO is discussed as the standard trust-region intuition. For the runnable VLM setting, the chapter emphasizes RLOO, where several completions are generated for the same prompt, each is scored, and each completion is compared to the average reward of the others:

$$b_i = \frac{1}{G-1} \sum_{j \neq i} r_j, \qquad A_i = r_i - b_i.$$

The simplified loss is

$$\mathcal{L}_{\mathrm{RLOO}}(\theta) = -\frac{1}{G} \sum_{i=1}^{G} A_i \log \pi_\theta(o_i \mid q).$$

This makes RLOO conceptually attractive because it uses a **relative baseline** for the same prompt and avoids some of the machinery of heavier PPO-style pipelines.

The broad lesson of the chapter is that adaptation methods differ along several axes at once:

- whether they change context or weights,
- whether they inject knowledge, demonstrations, labels, preferences, or rewards,
- whether they are temporary or persistent,
- whether they optimize imitation, preference ranking, or reward,
- and how much data, compute, and evaluation burden they require.

That is why the chapter ends with a budget ladder. Zero-shot and few-shot are cheap but limited. RAG adds grounding without changing the model. SFT and LoRA create reusable specialization. DPO aligns relative preferences. RL-based methods are the most expressive, but also the most expensive and easiest to misuse if the reward is poorly defined.

Sample Final Exam Questions

Question 1

Why is the chapter's adaptation ladder best understood as a sequence of **different intervention types** rather than a simple ranking from weak to strong methods?

Sample answer

The methods differ along multiple conceptual dimensions, not just "strength."
Zero-shot, few-shot, and RAG mainly change the **conditioning information** supplied at inference time, while SFT, LoRA, DPO, and RL-based methods change the **policy itself**.

This means the methods intervene at different levels:

- zero-shot changes only the instruction,
- few-shot changes the context by adding demonstrations,
- RAG changes the context by adding retrieved evidence,
- SFT changes the objective over labeled outputs,
- LoRA changes the parameterization of the update,
- DPO changes the supervision signal from demonstrations to preferences,
- RL-based methods optimize expected reward.

So the ladder is not merely a scale of "more powerful." It is a structured map of **what is being modified**: prompt, memory, labels, parameterization, preferences, or rewards. That is why the methods are often complementary rather than mutually exclusive.

Question 2

Why does the chapter emphasize adapting both attention projections and MLP projections in transformer-based VLMs?

Sample answer

The chapter highlights that different sublayers play different conceptual roles.
Attention projections such as W_Q, W_K, W_V, and W_O affect **routing**: they determine what information is queried, matched, attended, and reinserted into the residual stream. MLP projections such as `gate_proj`, `up_proj`, and `down_proj` affect **transformation**: they determine how information is nonlinearly processed after it arrives.

So adapting only attention changes how information flows, while adapting only the MLP changes how information is reshaped. Many downstream tasks need both:

- new visual-semantic associations,
- new token-to-token interaction patterns,
- and new local feature transformations.

That is why the chapter presents a broader adapter target set rather than treating LoRA as only an attention modification.

Question 3

How does the chapter reinterpret classical reinforcement-learning notation for autoregressive VLM generation?

Sample answer

The chapter shows that RLHF does not require abandoning the language-model viewpoint. It simply reinterprets token generation in RL terms:

- state s_t: prompt, image, and generated prefix,
- action a_t: next token,
- policy $\pi_\theta(a_t \mid s_t)$: next-token distribution,
- trajectory τ: full generated answer,
- reward: scalar score assigned to the answer.

This mapping matters because it explains why policy-gradient ideas apply naturally to VLMs. The model is still generating one token at a time, but now the sequence is judged by a reward function, human preference model, or verifier.

So RLHF is best seen as **policy optimization over token sequences conditioned on multimodal context**.

Question 4

Suppose a VLM gives well-formatted answers but repeatedly confuses closely related species because it lacks domain knowledge. Why would RAG often be a more principled first intervention than SFT or DPO?

Sample answer

If the primary failure is **missing knowledge**, then the problem is not yet one of answer style or preference alignment. RAG directly addresses the missing-knowledge bottleneck by injecting external evidence at inference time.

Conceptually, RAG changes the conditional distribution from

$$p_\theta(y \mid x, v)$$

to a retrieved-evidence version such as

$$p_{\theta,\eta}(y \mid x, v) = \sum_z p_\eta(z \mid q(x, v))\, p_\theta(y \mid x, v, z).$$

This is often more principled than immediate SFT because SFT teaches the model to imitate desired outputs, but it does not necessarily give it access to new, explicit supporting evidence at inference time. DPO is even less direct for this failure mode because it adjusts preferences among answers, not the factual evidence base itself.

So when the central issue is **grounding and missing task knowledge**, RAG is often the cleanest first upgrade.

Question 5

How can RAG, LoRA-based SFT, and RL-based alignment be viewed as operating on three different layers of adaptation within one overall system?

Sample answer

These methods operate at different levels of the stack.

RAG acts on the **evidence layer**. It decides what external information is available to the generator at inference time.

LoRA-based SFT acts on the **policy-parameter layer**. It changes the model's internal mapping from multimodal context to answer distribution, but does so through a low-rank parameterization.

RL-based alignment acts on the **behavioral objective layer**. It changes what the policy is rewarded for, typically using preference or reward signals rather than fixed gold outputs.

So one can think of the three as:

- RAG: change what the model sees,
- LoRA/SFT: change how the model maps seen information to outputs,
- RL alignment: change what output behavior is incentivized.

This layered view explains why the methods can be composed rather than treated as mutually exclusive alternatives.

Quantization Summary and Sample Questions

Concept-focused summary of quantization for deep learning and generative models

Quantization can be understood as a **numerical approximation problem** rather than merely a deployment trick. The starting point is a model with full-precision parameters and activations, together with the practical desire to reduce memory footprint, bandwidth, latency, and energy. Quantization introduces a controlled loss of numerical fidelity in exchange for these deployment benefits. The central question is not simply whether low precision is possible, but **where in the model lifecycle the approximation should be handled** and **which numerical objects are most important to preserve**.

A convenient starting point is the generic uniform quantizer,

$$\widehat{x} = s \cdot \mathrm{clip}\Big(\mathrm{round}\Big(\frac{x}{s}\Big), q_{\min}, q_{\max}\Big),$$

where s is the scale or step size, and $q_{\min}, q_{\max}$ determine the available integer codes. This formula already reveals the central tradeoff: the same scale controls both the **spacing between representable levels** and the **real-valued range** covered by those levels. If the scale is too small, the quantizer has fine resolution but severe clipping. If the scale is too large, the quantizer covers a wider range but becomes too coarse. Much of modern quantization can be read as a sequence of strategies for managing this precision-range tradeoff more intelligently.

A second foundational distinction is between **what is quantized** and **how it is quantized**. One may quantize weights, activations, or both; one may use per-tensor, per-channel, or group-wise scales; and one may keep some layers or operators at higher precision while quantizing others aggressively. This design space is already nontrivial before any training or calibration takes place. A useful way to organize it is into three phases: **pre-quantization**, **during-training quantization**, and **post-training quantization**. These are not isolated subfields, but three different times at which we may choose to handle the same underlying approximation problem.

In the **pre-quantization** phase, the central task is to understand the numerical structure of the model before low precision is imposed. Calibration is best viewed not as a mechanical preprocessing step but as a form of **distribution estimation**. One studies activation ranges, outliers, layer sensitivity, and the appropriate granularity of scales. Quantization error has at least two main sources: rounding error from a coarse grid and clipping error from an insufficient range. Pre-quantization therefore asks which tensors are hardest to compress, where outliers occur, and whether some layers should remain in higher precision. This phase is conceptually about **measurement, design, and anticipation**.

A more refined view of sensitivity comes from curvature. In one dimension, the loss increase caused by a quantization perturbation e is approximated by

$$\Delta \approx -\mathcal{L}'(w)e + \frac{1}{2}\mathcal{L}''(w)e^2.$$

Near a trained solution, the first derivative is often small, so the second derivative becomes the dominant local predictor of damage. In multiple dimensions, this becomes the Hessian-based approximation

$$\mathcal{L}(\widehat{W}) \approx \mathcal{L}(W) + \nabla\mathcal{L}(W)^\top(\widehat{W} - W) + \frac{1}{2}(\widehat{W} - W)^\top H(\widehat{W} - W).$$

The diagonal of the Hessian measures individual parameter sensitivity, while the off-diagonal terms measure coupling between parameters. This distinction helps explain why second-order post-training methods can do better than purely local rounding rules.

The **during-training** phase introduces quantization-aware training, where the quantizer is moved inside the learning loop. Instead of first learning the best floating-point model and only later compressing it, QAT learns a model that is already robust to quantization distortions. A generic objective is

$$\min_{\{W_\ell\},\,\Theta} \; \mathcal{L}\Big(f\big(x;\,\{Q_W(W_\ell)\}_{\ell=1}^{L},\,\{Q_a(a_\ell)\}_{\ell=1}^{L}\big)\Big),$$

so both weights and layer activations may be fake-quantized during training. The main difficulty here is not conceptual but differential: quantization contains rounding and clipping, both of which are hostile to ordinary gradient descent.

This is why the **straight-through estimator (STE)** matters so much. The forward pass uses the real fake-quantized values, but the backward pass substitutes a surrogate gradient, often acting like the identity inside the clipping range and zero outside it. The key idea is that QAT does not attempt to compute the exact derivative of a discrete operator; it instead uses a practical surrogate that points optimization in a useful direction. In that sense, QAT may be summarized as: **train under quantized behavior, but optimize with approximate gradients**.

Within the training-time family, two especially important learnable ideas are **PACT** and **LSQ**. PACT introduces a trainable clipping threshold α so that activations are no longer clipped by a fixed heuristic. Conceptually, this changes clipping from a manual engineering choice into a learned parameter that balances preservation of useful signal against the need for a narrower quantization range. LSQ goes one step further and learns the step size itself. In LSQ, the quantizer grid is no longer fixed beforehand; the spacing between levels becomes part of the optimization problem. A recurring theme appears here: better quantization often comes from making previously hand-chosen numerical decisions adaptive.

Binary and ternary training sit at the most aggressive end of the spectrum. Binary quantization keeps only sign information, while ternary quantization adds a zero state and therefore sparsity. These methods are useful not mainly because they dominate mainstream LLM deployment, but because they make the core approximation issue stark: as the representational alphabet becomes extremely small, the question becomes whether training can reorganize the network strongly enough to survive the compression.

The **post-training** phase starts from a pretrained model and asks how to quantize it as cheaply as possible without reopening full training. The baseline is simple **round-to-nearest (RTN)** quantization, which treats each weight independently. RTN is best viewed as the natural reference point rather than a straw man. Its value is that it cleanly exposes what more advanced PTQ methods are trying to fix: lack of data awareness, lack of layerwise reconstruction, and lack of sensitivity information.

From there the picture develops progressively richer PTQ ideas. **AdaRound** replaces naive nearest rounding by a data-driven layer reconstruction objective. Its central insight is that the best rounding direction for a given weight is not necessarily the one that minimizes scalar weight error; what matters is preservation of the **layer output** on calibration data. This shifts the viewpoint from "approximate each weight well" to "preserve the function of the layer well."

The next step is **second-order PTQ**, illustrated through GPTQ. Here the Hessian is no longer just a mathematical curiosity; it becomes the mechanism by which the algorithm predicts which perturbations are expensive. GPTQ quantizes a column or block, computes the induced error, and then updates the remaining unquantized weights using approximate inverse-curvature information so that the downstream distortion is reduced. The key conceptual point is that first-order information answers a training question — how to descend the loss — whereas PTQ faces a different question: **if quantization forces a perturbation, how harmful will that perturbation be?** That is why curvature naturally enters.

Transformer-specific PTQ then receives dedicated treatment. **SmoothQuant** is presented as an equivalent rescaling trick for models with activation outliers. By writing

$$y = Wx = (WS^{-1})(Sx),$$

the method migrates part of the numerical difficulty from activations into weights without changing the floating-point function of the layer. This illustrates a broader principle: sometimes the best quantization strategy is not to directly quantize a troublesome tensor, but to **reparameterize the computation** so that the troublesome tensor becomes easier to quantize.

AWQ and **OmniQuant** continue this theme in different ways. AWQ is activation-aware weight-only PTQ: activations reveal which weight channels matter most, so those channels receive special protection. OmniQuant stays in the PTQ regime but makes calibration itself more learnable through **Learnable Weight Clipping (LWC)** and **Learnable Equivalent Transformations (LET)**. Conceptually, OmniQuant sits between classical fixed-parameter PTQ and full QAT: it does not retrain the network end to end, but it also does not accept fixed clipping and scaling rules as final.

The notion of deployment broadens further with **I-BERT**. Quantizing only matrix multiplications is not enough if nonlinear operators such as GELU, Softmax, and LayerNorm remain floating point. The real deployment goal is a **fully quantized computation graph**, not merely quantized weight matrices. This is one of the most important conceptual shifts in the material, because it reframes quantization as a property of the entire executed graph, not just the stored parameters.

The final part turns to **generative and multimodal models**, where the same three-phase framework remains useful but the failure modes become more subtle. In **autoregressive LLMs**, difficulty comes from activation outliers, deep residual propagation, attention sensitivity, and KV-cache accumulation during long-context decoding. In **diffusion models**, small errors can compound across denoising timesteps, and calibration must often be timestep-aware because the activation distributions evolve through the reverse process. In **diffusion transformers**, both transformer outlier issues and timestep dependence appear simultaneously. In **flow matching** and related continuous-time models, quantization perturbs a time-dependent vector field and therefore can alter the integrated trajectory of the solver itself; the literature here is still comparatively sparse.

The multimodal treatment broadens the scope further. CLIP-like dual encoders are hard to quantize because the goal is not only local task accuracy but preservation of a **shared image-text embedding geometry**. Fusion-style multimodal transformers are harder still because the visual encoder, projector, fusion blocks, and language decoder do not have equal sensitivities. The final message is that quantization is deeply modality dependent, and the hardest models are often those where low-bit errors propagate through **iterative, cross-modal, or long-context structures**.

Overall, quantization can be understood as a hierarchy of ideas. The first layer is numerical: levels, scales, range, clipping, and rounding. The second layer is structural: which tensors, layers, and operations are most sensitive. The third layer is algorithmic: calibration, QAT, PTQ, reconstruction, curvature, and equivalent transformations. The fourth layer is systems-oriented: what it means to obtain a genuinely deployable low-precision computation graph. The unifying achievement is to connect these layers into one picture: quantization is the art of deciding **when**, **where**, and **how** to absorb approximation error so that the model remains functionally useful under severe numerical constraints.

Sample Final Exam Questions

Question 1

What is the central tradeoff between scale and range, and why does a single scale create a structural tension in uniform quantization?

Sample answer

In a uniform quantizer, the scale s determines both the spacing between adjacent levels and the total real-valued range covered by the available integer codes. This means that the same parameter controls two competing objectives. If s is small, the levels are close together and rounding error is reduced, but the representable range becomes narrow and clipping error grows. If s is large, clipping is reduced because the range widens, but the grid becomes coarse and rounding error increases.

So the tension is structural, not accidental. For a fixed bit-width, one cannot improve local resolution without shrinking global coverage, or expand coverage without coarsening the grid. Much of the can be read as a search for better ways to manage this precision-range tension, either by learning scales, changing granularity, using mixed precision, or restructuring the computation itself.

Question 2

Why is the straight-through estimator conceptually necessary for quantization-aware training?

Sample answer

The problem is that the forward quantizer contains operations such as rounding and clipping, and their true derivatives are unhelpful for gradient descent. Rounding has zero derivative almost everywhere, and clipping suppresses gradients outside the allowed range. If one backpropagated through the true quantizer directly, learning would often stall or become numerically meaningless.

The STE solves this by separating forward fidelity from backward usability. The forward pass still exposes the network to quantization distortions, but the backward pass uses a surrogate derivative that allows optimization to proceed. So the STE is conceptually necessary because QAT needs the model to experience low-precision behavior while still receiving gradients that are useful enough to train on.

Question 3

Why are PACT and LSQ both examples of the broader idea that good quantization often comes from learning previously fixed numerical decisions?

Sample answer

PACT and LSQ both convert what might otherwise be hand-designed quantizer settings into trainable parameters. In PACT, the clipping threshold α is learned from the task loss rather than fixed by heuristic range selection. In LSQ, the step size itself becomes a learned quantity rather than a predetermined scale chosen only from statistics.

The broader conceptual theme is that quantization is improved when the network is allowed to adapt not only its weights but also the numerical rules that define the quantizer. This is a strong shift from early quantization schemes, where clipping and scale were often treated as engineering constants. PACT and LSQ show that these constants are actually part of the optimization problem.

Question 4

Why does GPTQ use inverse-curvature compensation after quantizing a column or block, instead of simply rounding each block independently?

Sample answer

If one rounds each block independently, the induced error is accepted as final and may accumulate through the rest of the layer. GPTQ instead recognizes that once one column is quantized, the remaining unquantized columns are still free to move. Under a second-order approximation, those remaining degrees of freedom can be adjusted in a way that absorbs part of the damage caused by the newly quantized block.

The inverse Hessian appears because it gives the locally optimal compensating update under the quadratic loss model. So GPTQ is conceptually important not just because it is a low-bit PTQ method, but because it turns quantization into a sequence of **quantize, estimate damage, compensate, continue** rather than a sequence of isolated rounding events.

Question 5

Why are diffusion models harder to quantize post-training than ordinary feedforward classifiers?

Sample answer

A feedforward classifier usually experiences quantization error only across depth: a perturbed activation is propagated through later layers in one pass. A diffusion model is harder because the denoiser is called repeatedly across many timesteps. Small quantization errors can therefore compound over the denoising trajectory, not just inside one forward pass.

A second complication is that diffusion statistics vary with timestep. This means one static calibration rule may be poor across the entire reverse process. Thus diffusion PTQ is hard for two linked reasons: the model is iterative, and its operating distribution shifts over time.